

可以。既然你现在准备自己先手工实现，而这次又明确让我“实现核心程序功能”，我建议我们**不要一下子把 Compose 页面、设置页、动画全部塞进来**，而是先把最关键的后台可靠性链路完整实现出来：

Room → AlarmManager → BroadcastReceiver → Notification → BOOT_COMPLETED → 多 Timer

UI 只做一个最简单的入口，方便你测试。

下面这套代码我按你熟悉的 **Kotlin + Room + Hilt + ViewModel + Compose** 来组织。

另外我根据目前 Android 官方文档核了一下几个关键点：Android 12+ 使用 exact alarm 需要处理 `SCHEDULE_EXACT_ALARM`；Android 14+ 新安装的多数应用这个权限默认是拒绝的，所以代码不能假定它一定存在。[\(Android Developers\)](#) Android 13+ 通知还需要 `POST_NOTIFICATIONS`。[\(Android Developers\)](#)

一、整体结构

先建立下面这些类：

```
data/
├── TimerEntity.kt
├── TimerDao.kt
└── AppDatabase.kt

domain/
└── TimerRepository.kt

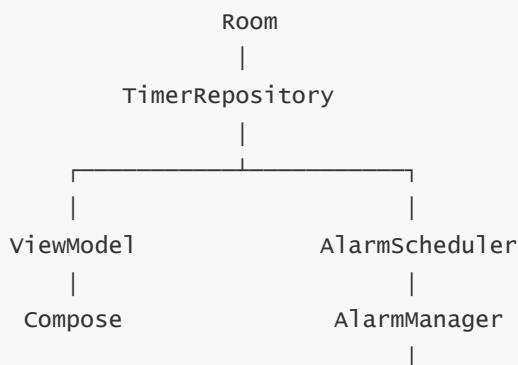
alarm/
├── AlarmScheduler.kt
├── TimerAlarmReceiver.kt
└── BootReceiver.kt

notification/
└── TimerNotificationManager.kt

di/
└── AppModule.kt

ui/
└── TimerViewModel.kt
```

核心关系：



▼
TimerAlarmReceiver
|
Notification

二、TimerEntity

这里我建议 `id` 直接使用 `String` + UUID。

```
@Entity(  
    tableName = "timers"  
)  
data class TimerEntity(  
  
    @PrimaryKey  
    val id: String,  
  
    /**  
     * 用户给这个倒计时起的名字  
     */  
    val title: String,  
  
    /**  
     * 开始时间, System.currentTimeMillis()  
     */  
    val startTime: Long,  
  
    /**  
     * 绝对结束时间  
     */  
    val endTime: Long,  
  
    /**  
     * 当前状态  
     */  
    val status: TimerStatus  
)  
  
enum class TimerStatus {  
    RUNNING,  
    PAUSED,  
    FINISHED,  
    CANCELLED  
}
```

这里最重要的字段其实只有：

```
val endTime: Long
```

它才是倒计时真正的时间依据。

三、TimerDao

```
@Dao
interface TimerDao {

    @Query(
        """
        SELECT * FROM timers
        WHERE status = 'RUNNING'
        ORDER BY endTime ASC
        """
    )
    fun observeRunningTimers(): Flow<List<TimerEntity>>

    @Query("SELECT * FROM timers WHERE id = :id LIMIT 1")
    suspend fun getById(id: String): TimerEntity?

    @Query(
        """
        SELECT * FROM timers
        WHERE status = 'RUNNING'
        """
    )
    suspend fun getRunningTimers(): List<TimerEntity>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(timer: TimerEntity)

    @Update
    suspend fun update(timer: TimerEntity)

    @Query(
        """
        UPDATE timers
        SET status = :status
        WHERE id = :id
        """
    )
    suspend fun updateStatus(
        id: String,
        status: TimerStatus
    )

    @Query("DELETE FROM timers WHERE id = :id")
    suspend fun delete(id: String)
}
```

四、Room Database

```
@Database(
    entities = [TimerEntity::class],
    version = 1,
    exportSchema = true
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun timerDao(): TimerDao
}
```

五、Hilt 注入 Room

```
@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides
    @Singleton
    fun provideDatabase(
        @ApplicationContext context: Context
    ): AppDatabase {
        return Room.databaseBuilder(
            context,
            AppDatabase::class.java,
            "timer.db"
        ).build()
    }

    @Provides
    fun provideTimerDao(
        database: AppDatabase
    ): TimerDao {
        return database.timerDao()
    }
}
```

六、AlarmScheduler

这是整个程序最关键的类之一。

```
@Singleton
class AlarmScheduler @Inject constructor(
    @ApplicationContext private val context: Context
) {
```

```

private val alarmManager =
    context.getSystemService(AlarmManager::class.java)

fun canScheduleExactAlarms(): Boolean {
    return if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
        alarmManager.canScheduleExactAlarms()
    } else {
        true
    }
}

fun schedule(timer: TimerEntity) {

    check(canScheduleExactAlarms()) {
        "Exact alarm permission is not granted."
    }

    val intent = Intent(
        context,
        TimerAlarmReceiver::class.java
    ).apply {
        action = ACTION_TIMER_ALARM
        putExtra(EXTRA_TIMER_ID, timer.id)
    }

    val pendingIntent = PendingIntent.getBroadcast(
        context,
        requestCode(timer.id),
        intent,
        PendingIntent.FLAG_UPDATE_CURRENT or
            PendingIntent.FLAG_IMMUTABLE
    )

    val triggerAt = timer.endTime

    AlarmManagerCompat.setExactAndAllowWhileIdle(
        alarmManager,
        AlarmManager.RTC_WAKEUP,
        triggerAt,
        pendingIntent
    )
}

fun cancel(timerId: String) {

    val intent = Intent(
        context,
        TimerAlarmReceiver::class.java
    ).apply {
        action = ACTION_TIMER_ALARM
        putExtra(EXTRA_TIMER_ID, timerId)
    }
}

```

```

        val pendingIntent = PendingIntent.getBroadcast(
            context,
            requestCode(timerId),
            intent,
            PendingIntent.FLAG_UPDATE_CURRENT or
                PendingIntent.FLAG_IMMUTABLE
        )

        alarmManager.cancel(pendingIntent)

        pendingIntent.cancel()
    }

    private fun requestCode(timerId: String): Int {
        return timerId.hashCode()
    }

    companion object {

        const val ACTION_TIMER_ALARM =
            "com.example.timer.ACTION_TIMER_ALARM"

        const val EXTRA_TIMER_ID =
            "timer_id"
    }
}

```

这里使用：

```
AlarmManagerCompat.setExactAndAllowWhileIdle()
```

它的特点正是**精确触发**，同时允许在 **Doze/idle 状态下触发**。([Android Developers](#))

七、为什么 requestCode 很重要

你之前问过多倒计时。

这里：

```
requestCode(timer.id)
```

就是让：

```

Timer A → PendingIntent A
Timer B → PendingIntent B
Timer C → PendingIntent C

```

互不冲突。

例如：

```
UUID A → hashCode = 12345
UUID B → hashCode = 67890
UUID C → hashCode = 34567
```

这样 AlarmManager 可以同时存在多个 alarm。

八、NotificationManager

```
@Singleton
class TimerNotificationManager @Inject constructor(
    @ApplicationContext private val context: Context
) {

    private val notificationManager =
        context.getSystemService(
            NotificationManager::class.java
        )

    init {
        createChannel()
    }

    private fun createChannel() {

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {

            val channel = NotificationChannel(
                CHANNEL_ID,
                "倒计时提醒",
                NotificationManager.IMPORTANCE_HIGH
            ).apply {
                description = "倒计时结束提醒"
            }

            notificationManager.createNotificationChannel(
                channel
            )
        }
    }

    fun showTimerFinished(timer: TimerEntity) {

        val notification = NotificationCompat.Builder(
            context,
            CHANNEL_ID
        )

        .setSmallIcon(R.drawable.ic_notification)
        .setContentTitle(timer.title)
        .setContentText("倒计时结束")
        .setPriority(NotificationCompat.PRIORITY_HIGH)
```

```

        .setAutoCancel(true)
        .build()

        notificationManager.notify(
            timer.id.hashCode(),
            notification
        )
    }

    companion object {
        const val CHANNEL_ID = "timer_finished"
    }
}

```

Android 8.0+ 必须使用 Notification Channel，而且重复创建同一个 channel 是安全的。([Android Developers](#))

这里我特意使用：

```
timer.id.hashCode()
```

作为 notification ID。

所以：

```

Timer A → Notification A
Timer B → Notification B
Timer C → Notification C

```

多个倒计时同时结束时不会互相覆盖。

九、TimerAlarmReceiver

这是 AlarmManager 到时间以后真正执行的地方。

```

@AndroidEntryPoint
class TimerAlarmReceiver : BroadcastReceiver() {

    @Inject
    lateinit var timerDao: TimerDao

    @Inject
    lateinit var notificationManager: TimerNotificationManager

    @Inject
    lateinit var alarmScheduler: AlarmScheduler

    override fun onReceive(
        context: Context,
        intent: Intent
    ) {

```



```

val timerId =
    intent.getStringExtra(
        AlarmScheduler.EXTRA_TIMER_ID
    ) ?: return

val pendingResult = goAsync()

CoroutineScope(Dispatchers.IO).launch {

    try {

        val timer = timerDao.getById(timerId)

        if (timer == null) {
            return@launch
        }

        /*
         * 非 RUNNING 状态说明:
         *
         * 可能已经被用户取消
         * 或者这个 alarm 是旧的
         */
        if (timer.status != TimerStatus.RUNNING) {
            return@launch
        }

        /*
         * 更新数据库状态
         */
        timerDao.updateStatus(
            timerId,
            TimerStatus.FINISHED
        )

        /*
         * 发通知
         */
        notificationManager.showTimerFinished(
            timer
        )

    } finally {
        pendingResult.finish()
    }
}
}
}

```

这里有个很重要的设计：

Receiver 收到 Alarm 后，不能盲目发通知，而应该再次查询数据库确认 Timer 是否仍然 RUNNING。

为什么？

例如：

```
10:00 创建 Timer
10:05 设置 Alarm
10:10 用户取消 Timer
10:10 cancel Alarm
```

理论上 Alarm 已经取消。

但为了处理各种竞态和旧 PendingIntent，我们仍然让 Receiver 自己再验证一次。

这样更加可靠。

十、Repository

```
@Singleton
class TimerRepository @Inject constructor(
    private val timerDao: TimerDao,
    private val alarmsScheduler: AlarmScheduler
) {

    fun observeRunningTimers(): Flow<List<TimerEntity>> {
        return timerDao.observeRunningTimers()
    }

    suspend fun createTimer(
        title: String,
        durationMillis: Long
    ): TimerEntity {

        require(durationMillis > 0) {
            "durationMillis must be greater than zero"
        }

        val now = System.currentTimeMillis()

        val timer = TimerEntity(
            id = UUID.randomUUID().toString(),
            title = title,
            startTime = now,
            endTime = now + durationMillis,
            status = TimerStatus.RUNNING
        )

        timerDao.insert(timer)

        alarmsScheduler.schedule(timer)

        return timer
    }
}
```

```
suspend fun cancelTimer(
    timerId: String
) {

    alarmsScheduler.cancel(timerId)

    timerDao.updateStatus(
        timerId,
        TimerStatus.CANCELLED
    )
}
}
```

这里形成了一个非常清晰的职责：

```
createTimer()
|
├─ Room.insert()
|
└─ AlarmManager.schedule()
```

十一、这里有一个小的事务问题

你可能已经注意到了：

```
timerDao.insert(timer)
alarmsScheduler.schedule(timer)
```

这两个操作不是一个真正的数据库事务。

实际上这里**不能简单地使用 Room transaction 把两者包进去**，因为 AlarmManager 不是数据库操作。

正确思路反而是：

1. Timer 写入 Room
2. Alarm 注册

如果第 2 步失败：

```
Room 有 Timer
Alarm 没有
```

我们仍然可以在 App 启动时：

```
查询所有 RUNNING Timer
重新 schedule
```

因此下一步我会专门做一个：

用于启动时重新同步。

这样系统的可靠性更强。

十二、BootReceiver

手机重启以后重新注册 Alarm。

```
@AndroidEntryPoint
class BootReceiver : BroadcastReceiver() {

    @Inject
    lateinit var timerDao: TimerDao

    @Inject
    lateinit var alarmScheduler: AlarmScheduler

    override fun onReceive(
        context: Context,
        intent: Intent
    ) {

        if (
            intent.action != Intent.ACTION_BOOT_COMPLETED &&
            intent.action != Intent.ACTION_MY_PACKAGE_REPLACED
        ) {
            return
        }

        val pendingResult = goAsync()

        CoroutineScope(Dispatchers.IO).launch {

            try {

                val timers =
                    timerDao.getRunningTimers()

                val now = System.currentTimeMillis()

                timers.forEach { timer ->

                    if (timer.endTime <= now) {

                        timerDao.updateStatus(
                            timer.id,
                            TimerStatus.FINISHED
                        )

                    }

                }

            }

        }

    }

}
```

```

        } else {

            alarmScheduler.schedule(timer)

        }
    }

    } finally {
        pendingResult.finish()
    }
}
}
}
}

```

这里还顺便处理：

```
ACTION_MY_PACKAGE_REPLACED
```

也就是 App 更新以后重新注册。

十三、Manifest

```

<manifest ...>

    <!-- Android 12+ exact alarm -->
    <uses-permission
        android:name="android.permission.SCHEDULE_EXACT_ALARM" />

    <!-- Android 13+ notification -->
    <uses-permission
        android:name="android.permission.POST_NOTIFICATIONS" />

    <!-- 手机重启 -->
    <uses-permission
        android:name="android.permission.RECEIVE_BOOT_COMPLETED" />

    <application
        ...>

        <receiver
            android:name=".alarm.TimerAlarmReceiver"
            android:exported="false" />

        <receiver
            android:name=".alarm.BootReceiver"
            android:exported="true">

            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" />
                <action android:name="android.intent.action.MY_PACKAGE_REPLACED" />
            </intent-filter>

```

```
        </receiver>

    </application>

</manifest>
```

Android 官方目前明确要求 Android 12+ 使用 exact alarm 时处理 `SCHEDULE_EXACT_ALARM`，而 Android 14 起它对符合条件的新安装应用默认不再预授权。([Android Developers](#))

十四、ViewModel

现在 ViewModel 就非常干净了。

```
@HiltViewModel
class TimerViewModel @Inject constructor(
    private val repository: TimerRepository,
    private val alarmScheduler: AlarmScheduler
) : ViewModel() {

    val timers: StateFlow<List<TimerEntity>> =
        repository
            .observeRunningTimers()
            .stateIn(
                scope = viewModelScope,
                started = SharingStarted.WhileSubscribed(5_000),
                initialValue = emptyList()
            )

    fun createTimer(
        title: String,
        durationMillis: Long
    ) {

        viewModelScope.launch {

            repository.createTimer(
                title = title,
                durationMillis = durationMillis
            )
        }
    }

    fun cancelTimer(
        timerId: String
    ) {

        viewModelScope.launch {

            repository.cancelTimer(
                timerId
            )
        }
    }
}
```

```

    )
}

fun canScheduleExactAlarms(): Boolean {
    return alarmScheduler.canScheduleExactAlarms()
}
}

```

注意这里已经没有 `CountDownTimer` 了。

十五、Compose UI 怎么显示剩余时间

这里反而可以使用一个普通的 UI ticker。

例如：

```

@Composable
fun TimerItem(
    timer: TimerEntity
) {

    var now by remember {
        mutableLongStateOf(
            System.currentTimeMillis()
        )
    }

    LaunchedEffect(Unit) {

        while (true) {

            now = System.currentTimeMillis()

            delay(1000)
        }
    }

    val remaining =
        (timer.endTime - now)
            .coerceAtLeast(0)

    val seconds =
        remaining / 1000

    val minutes =
        seconds / 60

    val second =
        seconds % 60

```

```
        Text(  
            text = "%02d:%02d".format(  
                minutes,  
                second  
            )  
        )  
    }  
}
```

这个时候即使 UI ticker 被：

Activity销毁
App进入后台
进程被杀

都完全没关系。

因为：

UI ticker

只是：

“告诉用户还剩多少时间。”

它已经不承担：

“负责到时间提醒。”

十六、Exact Alarm 权限

这个一定要做。

例如：

```
fun requestExactAlarmPermission(  
    context: Context  
) {  
  
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {  
  
        val intent = Intent(  
            Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM,  
            Uri.parse(  
                "package:${context.packageName}"  
            )  
        )  
  
        context.startActivity(intent)  
    }  
}
```


创建 Timer 前：

```
if (!viewModel.canScheduleExactAlarms()) {  
  
    requestExactAlarmPermission(context)  
  
    return  
}
```

Android 官方建议调用 `canScheduleExactAlarms()` 判断权限，并在没有权限时跳转到系统的 exact alarm 设置页。([Android Developers](#))

十七、通知权限

Android 13+:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {  
  
    requestPermissions(  
        arrayOf(  
            Manifest.permission.POST_NOTIFICATIONS  
        ),  
        REQUEST_NOTIFICATION  
    )  
}
```

否则即使：

```
AlarmManager  
  ↓  
Receiver  
  ↓  
NotificationManager
```

整个链条都正常，用户还是看不到通知。Android 13 起 `POST_NOTIFICATIONS` 是运行时权限。([Android Developers](#))

十八、一个非常重要的改进：App 启动时也恢复 Alarm

我建议不要只依赖 `BootReceiver`。

App 每次启动都执行一次：

```
Room
↓
查询 RUNNING Timer
↓
重新 schedule Alarm
```

因为这样可以处理更多异常情况。

例如：

```
Timer存在
Alarm丢失
```

App 一启动：

```
重新注册
```

因此再增加：

```
@Singleton
class AlarmRestoreManager @Inject constructor(
    private val timerDao: TimerDao,
    private val alarmScheduler: AlarmScheduler
) {

    suspend fun restore() {

        val timers = timerDao.getRunningTimers()

        val now = System.currentTimeMillis()

        timers.forEach { timer ->

            if (timer.endTime <= now) {

                timerDao.updateStatus(
                    timer.id,
                    TimerStatus.FINISHED
                )

            } else {

                alarmScheduler.schedule(timer)
            }
        }
    }
}
```

然后 Application：

```
@HiltAndroidApp
class TimerApplication : Application()
```

在启动 Activity / ViewModel 初始化时调用一次即可。

十九、不过这里出现一个值得你自己思考的问题

假设：

```
Timer A
endTime = 20:00
```

然后：

```
19:50
App启动
```

AlarmRestoreManager：

```
schedule(A)
```

没有问题。

但是如果：

```
20:00
Alarm触发
```

Receiver：

```
status = FINISHED
```

然后：

```
20:01
App又启动
```

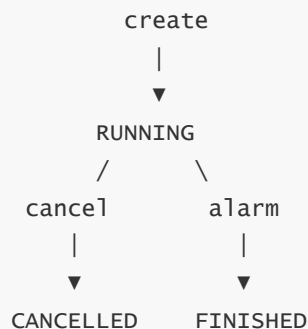
查询：

```
RUNNING Timer
```

已经没有 A。

也没问题。

所以整个状态机是自治的：

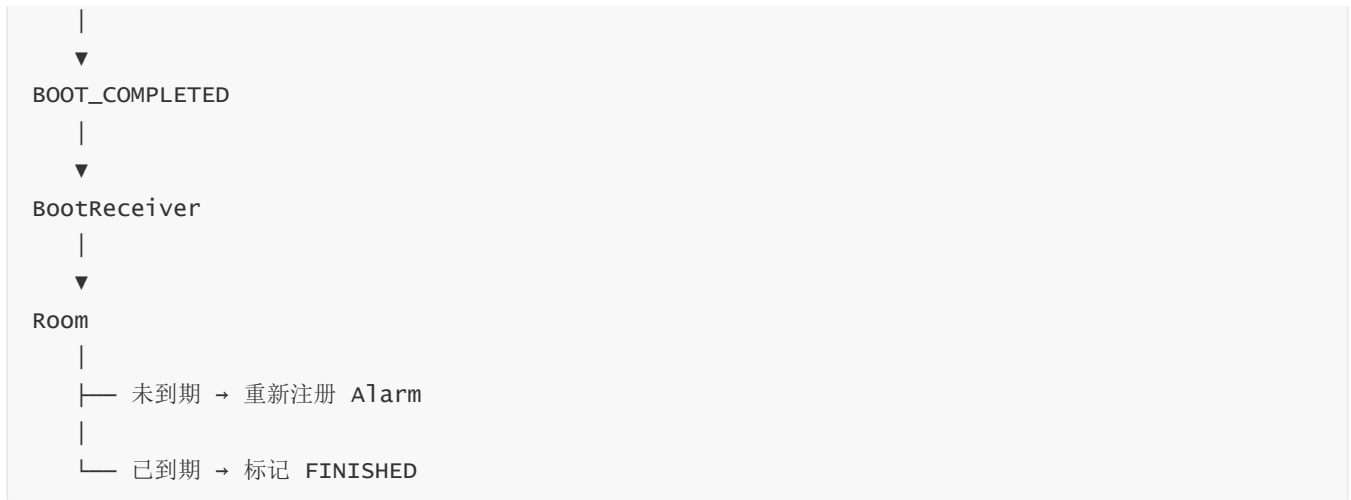


二十、整个核心链路最终就是这样



手机重启则是：

手机重启



我建议你现在先不要继续往 UI 扩展

先把上面这一套核心链路跑通，然后用 adb 主动杀进程、锁屏、Doze、重启手机测试它。

特别是先验证这 6 个场景：

- ① 前台运行
- ② 后台运行
- ③ 从最近任务划掉
- ④ adb 强制停止/杀进程后的行为
- ⑤ Doze
- ⑥ 手机重启

其中有一个测试要特别注意：“从设置里强制停止 App (Force Stop)”和“系统杀掉 App 进程”不是一回事。Android 对 force-stop 后的行为有额外限制，不能把它简单当作普通进程被杀来测试。

另外，我这里故意没有把 WorkManager 塞进链路。对于“到某个精确时间给用户发提醒”这个核心任务，AlarmManager + PendingIntent + BroadcastReceiver 更直接；如果 alarm 触发后要做长时间工作，再从 Receiver 转交 WorkManager。Android 官方也是这样区分的。[\(Android Developers\)](#)

还有一个我建议你下一步一定要改的地方：*** 上面的 BootReceiver 中“开机后发现 endTime <= now 就只标记 FINISHED”还不够完善——如果用户在手机关机期间错过了倒计时，正常的产品行为应该是“开机后补发一次“倒计时已结束”的通知。**这个涉及“错过的 Alarm 如何补偿”和“如何避免重复通知”的状态设计。这个做好之后，这套多倒计时核心架构就比较完整了。